

Multiple Linear, Ridge & Lasso Regression

One Dataset. Three Models. One ML Workflow.

Iris dataset · Python · Scikit-Learn

Meet the Iris Dataset	2	Scikit-Learn Map	3
Define X and y	4	Split + Scale	5
Multiple Linear Regression	6	Collinearity: The Trap	7
Ridge (L2)	8	Lasso (L1)	9
Coefficient Path	10	MSE, MAE, R ² + Result	11
Which One to Use?	12	Complete Code	13

Meet the Iris Dataset

150 flowers. 4 numeric measurements each. We turn it into a real regression problem.

X (predictors)

sepal width
petal length
petal width

→ Model →

y (target)

sepal length

Why not predict species? It is a category. Linear regression needs a continuous target, so we predict **sepal length (cm)**.

3 predictors → 1 target is what makes this **Multiple** Linear Regression.

TIP · টিপস

মনে রাখো: species একটা category, তাই regression-এ এটা target হতে পারে না। এই কারণেই আমরা continuous মান sepal length-কে target বানিয়েছি।

Scikit-Learn Map

Each tool lives in its own sklearn module. Learn this map once.

```
sklearn
├── datasets
│   └── load_iris
├── model_selection
│   ├── train_test_split
│   ├── cross_val_score
│   └── KFold
├── preprocessing
│   └── StandardScaler
├── pipeline
│   └── make_pipeline
├── linear_model
│   ├── LinearRegression
│   ├── Ridge
│   └── Lasso
├── metrics
│   ├── mean_squared_error
│   ├── mean_absolute_error
│   └── r2_score
```

■ library / sklearn names ■ modules & methods ■ your variables
■ text ■ numbers

TIP · টিপস

প্রতিটা tool sklearn-এর আলাদা module-এ থাকে। এই map-টা একবার বুঝে নিলে import নিয়ে আর ঝামেলা হবে না।

Define X and y

```
import pandas as pd
from sklearn.datasets import load_iris

iris = load_iris()
df = pd.DataFrame(iris.data,
                  columns=iris.feature_names)

X = df.drop(columns="sepal length (cm)")
y = df["sepal length (cm)"]
```

X = 3 columns (sepal width, petal length, petal width)
y = 1 column (sepal length)

TIP · টিপস

X বড় হাতের কারণ এটা table (একাধিক column), আর y ছোট হাতের কারণ এটা একটাই column। এই convention-টা সব জায়গায় চলে।

Train/Test Split, then Scale

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42)

scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)
```

Train 80% · 120 rows

Test 20% · 30

fit_transform on train, only **transform** on test. The scaler must never learn from test data.

TIP · টিপস

Scaler শুধু train data-তে fit করো, test data-তে শুধু transform করো। test data-র তথ্য আগে থেকে দেখে ফেললে data leakage হয়।

Multiple Linear Regression

$$\hat{y} = \beta_0 + \beta_1(\text{sepal width}) + \beta_2(\text{petal length}) + \beta_3(\text{petal width})$$

```
from sklearn.linear_model import LinearRegression

lin = LinearRegression()
lin.fit(X_train_s, y_train)
pred = lin.predict(X_test_s)
```

Feature	Linear (scaled)
sepal width	+0.297
petal length	+1.322
petal width	-0.505

Intercept = 5.809 · Test R^2 = 0.852

TIP · টিপস

Scaled data-তে coefficient মানে, feature এক standard deviation বাড়লে sepal length কতটা বাড়ে। তাই এখন coefficient-গুলো নিজেদের মধ্যে তুলনা করা যায়।

Collinearity: The Hidden Trap

```
print(X.corr().round(2))
```

	sepal width	petal length	petal width
sepal width	1.00	-0.43	-0.37
petal length	-0.43	1.00	0.96
petal width	-0.37	0.96	1.00

petal length ↔ petal width: $r = 0.96$

They carry almost the same information.

Alone, petal width and sepal length move together ($r = 0.82$). Yet Linear Regression gives petal width a **negative** coefficient: **-0.505**. The sign flipped.

TIP · টিপস

Model-এর কাছে petal length আর petal width প্রায় একই তথ্য। তাই কে কতটা credit পাবে সেটা সে ঠিকমতো ভাগ করতে পারে না, আর একটার sign উল্টে যায়।

Ridge Regression (L2)

$$\text{Loss} = \text{RSS} + \alpha \cdot \sum \beta_j^2$$

```
from sklearn.linear_model import Ridge

ridge = Ridge(alpha=10)
ridge.fit(X_train_s, y_train)
ridge_pred = ridge.predict(X_test_s)
```

Feature	Linear	Ridge
sepal width	+0.297	+0.212
petal length	+1.322	+0.636
petal width	-0.505	+0.114

Ridge pulls petal width from **-0.505** to **+0.114**. The sign is sensible again. Larger alpha = stronger shrinkage.

TIP · টিপস

Ridge সব coefficient-কে ছোট করে আনে, কিন্তু কোনোটাকে পুরোপুরি শূন্য করে না। Correlated feature থাকলে এটাই সবচেয়ে নিরাপদ।

Lasso Regression (L1)

$$\text{Loss} = \text{RSS} + \alpha \cdot \sum |\beta_j|$$

```
from sklearn.linear_model import Lasso

lasso = Lasso(alpha=0.05)
lasso.fit(X_train_s, y_train)
lasso_pred = lasso.predict(X_test_s)
```

Feature	Linear	Ridge	Lasso
sepal width	+0.297	+0.212	+0.180
petal length	+1.322	+0.636	+0.735
petal width	-0.505	+0.114	+0.000

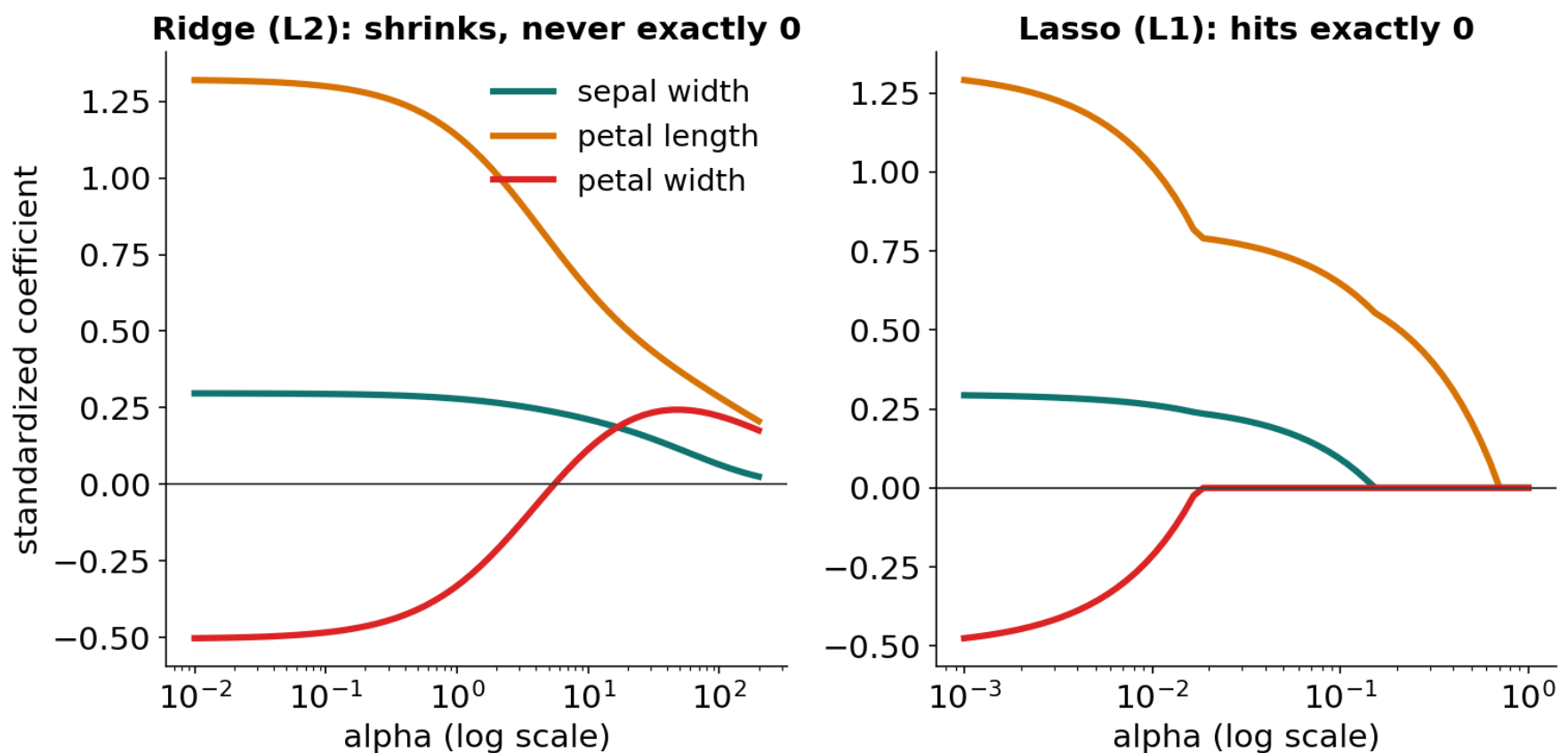
Lasso sets petal width to **exactly 0.000**. It dropped the redundant feature by itself.

Note: sklearn's Lasso divides the error term by $2n$ and Ridge does not, so the same alpha means different strength in each.

TIP · টিপস

Lasso কিছু coefficient-কে ঠিক শূন্য করে দেয়। মানে সে নিজেই বলে দেয় কোন feature বাদ দিলেও চলবে।

Watch the Coefficients Shrink



Ridge glides toward zero but never lands on it. **Lasso** snaps features to exactly zero one by one. Petal width goes first.

TIP · টিপস

Alpha বাড়ালে coefficient-গুলো শূন্যের দিকে যায়। সঠিক alpha বাছাই করতে cross-validation ব্যবহার করো (যেমন RidgeCV, LassoCV)।

MSE, MAE, R^2 and the Honest Result

Metric	Meaning
MSE	Average squared error (punishes big misses)
MAE	Average absolute error (in cm)
R^2	Share of variation explained

Model	MSE	MAE	Test R^2	5-fold CV R^2
Linear	0.102	0.234	0.852	0.840 ± 0.036
Ridge	0.097	0.225	0.860	0.795 ± 0.063
Lasso	0.097	0.222	0.859	0.808 ± 0.050

On one 30-row test set, Ridge and Lasso look slightly better. With shuffled 5-fold CV, plain Linear is actually best. **The three models are basically tied here.**

Regularization did not win on accuracy. It won on **stable, sensible coefficients.**

TIP · টিপস

Test set মাত্র 30 row, তাই R^2 -এর ছোট পার্থক্য দেখে winner ঠিক কোরো না। Shuffled cross-validation দিয়ে আবার যাচাই করো।

Which One Should You Use?

Linear

Start here. Your baseline. Fine when features are few and not strongly correlated.

Ridge

Features are correlated. Keeps all of them, makes coefficients stable.

Lasso

Many features. Automatically drops weak or redundant ones.

Pipeline in one line: Dataset → X, y → Split → Scale → Fit 3 models → Compare with CV

TIP · টিপস

- ১) সবসময় simple Linear Regression দিয়ে শুরু করো, সেটাই তোমার baseline।
- ২) Feature পরস্পর correlated হলে Ridge ব্যবহার করো।
- ৩) Feature অনেক বেশি হলে Lasso দিয়ে অপ্রয়োজনীয়গুলো বাদ দাও।

Complete Code (Part 1)

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import (
    train_test_split, cross_val_score, KFold)
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.linear_model import (
    LinearRegression, Ridge, Lasso)
from sklearn.metrics import (
    mean_squared_error, mean_absolute_error, r2_score)

# 1. Data
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
X = df.drop(columns="sepal length (cm)")
y = df["sepal length (cm)"]
print(X.corr().round(2))

# 2. Split, then scale (fit on train only)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42)
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)
```

TIP · টিপস

পুরো code একটা file-এ copy করে চালালেই হবে। Part 1-এ data তৈরি আর split/scale, Part 2-তে তিনটা model।

Complete Code (Part 2)

```
# 3. Three models on the same scaled data
models = {
    "Linear": LinearRegression(),
    "Ridge": Ridge(alpha=10),
    "Lasso": Lasso(alpha=0.05),
}
rows, coefs = [], {}
for name, model in models.items():
    model.fit(X_train_s, y_train)
    pred = model.predict(X_test_s)
    coefs[name] = model.coef_
    rows.append({
        "Model": name,
        "MSE": mean_squared_error(y_test, pred),
        "MAE": mean_absolute_error(y_test, pred),
        "R2": r2_score(y_test, pred)})
print(pd.DataFrame(rows).round(3))
print(pd.DataFrame(coefs, index=X.columns).round(3))

# 4. Shuffled 5-fold CV (fairer than one split)
cv = KFold(n_splits=5, shuffle=True, random_state=42)
for name, model in models.items():
    pipe = make_pipeline(StandardScaler(), model)
    score = cross_val_score(pipe, X, y, cv=cv, scoring="r2")
    print(name, "CV R2:", score.mean().round(3))
```

TIP · টিপস

models dictionary ব্যবহার করলে loop দিয়ে একই code-এ তিনটা model চালানো যায়। নতুন model যোগ করাও সহজ।